

Analiza natężenia ruchu pieszego w Melbourne na podstawie danych z systemu sensorów

Zespół:

Daria Bartkowiak Oliwia Wójcicka Karolina Kawulska

Czerwiec 2026

Autorstwo: model galaktyki i baza SQL – Oliwia Wójcicka;
proces ETL – Daria Bartkowiak; raporty Power BI – Karolina Kawulska.

1 Cel biznesowy projektu i korzyści

Celem projektu jest zbudowanie rozwiązania analitycznego, które pozwala monitorować i analizować natężenie ruchu pieszego w Melbourne. System umożliwia śledzenie liczby pieszych w ujęciu godzinowym, dziennym, tygodniowym, miesięcznym i rocznym oraz porównywanie aktywności pomiędzy różnymi lokalizacjami sensorów w mieście.

Odbiorcami rozwiązania są władze miasta oraz planiści przestrzenni. Rozwiązanie wspiera ich w podejmowaniu decyzji w takich obszarach jak:

- planowanie infrastruktury miejskiej,
- organizacja wydarzeń i zarządzanie ruchem w centrum,
- ocena atrakcyjności poszczególnych lokalizacji,
- analiza sposobu wykorzystania przestrzeni publicznej o różnych porach.

System ma odpowiadać na kluczowe pytania biznesowe:

- Które lokalizacje mają największy ruch pieszy?
- W jakich godzinach i dniach tygodnia ruch osiąga szczyty?
- Jak ruch zmienia się sezonowo i jakie ma długoterminowe trendy?
- Jak pogoda (temperatura, opady) wpływa na liczbę pieszych?

2 Opis źródeł danych

W projekcie wykorzystano trzy źródła danych.

1. Pedestrian Counting System (counts per hour) – główne źródło z oficjalnego portalu open data miasta Melbourne. Zawiera godzinowe dane o liczbie pieszych rejestrowanych przez sensory od 2009 roku. Dane są aktualizowane co miesiąc.

2. Pedestrian Counting System – Sensor Locations – źródło uzupełniające z tego samego portalu. Zawiera informacje o sensorach: status, lokalizację i kierunki pomiaru. Pole identyfikatora sensora łączy je ze zliczeniami pieszych. Część sensorów była przenoszona, usuwana lub czasowo wyłączana, co trzeba uwzględnić przy analizie historii.

3. Open-Meteo Historical Weather API – źródło danych pogodowych dla Melbourne: temperatura, opady, zachmurzenie, prędkość wiatru i kod pogody w ujęciu godzinowym. Dane pobierane są przez REST API w formacie JSON i łączone z ruchem pieszym po dacie, godzinie i lokalizacji.

Format i rozmiar. Dane o pieszych i sensorach mają postać tabelaryczną (pliki CSV lub zapytania do API), a dane pogodowe – format JSON. Zbiór należy ocenić jako średni: obejmuje okres od 2009 roku, ma rozdzielczość godzinową i wiele punktów pomiarowych.

Dostęp i częstotliwość odświeżania. Dane pobierane są z publicznych źródeł i odświeżane zgodnie z poniższą tabelą.

Źródło	Format	Częstotliwość
Zliczenia pieszych	CSV / API	miesięcznie
Lokalizacje sensorów	CSV / API	miesięcznie
Dane pogodowe (Open-Meteo)	JSON / API	miesięcznie (opcjonalnie częściej)

Linki do źródeł:

- <https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-monthly-counts-per-hour/>
- <https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-sensor-locations/>
- <https://open-meteo.com/en/docs/historical-weather-api>

3 Architektura rozwiązania

Rozwiązanie składa się z czterech warstw:

1. **Warstwa źródeł danych** – pliki CSV z portalu Melbourne oraz dane pogodowe z API Open-Meteo.
2. **Warstwa ETL** – proces w SQL Server Integration Services (SSIS), który pobiera dane, ładuje je do warstwy staging, waliduje i czyści, a następnie ładuje wymiary i tabele faktów.
3. **Warstwa hurtowni danych** – baza w Microsoft SQL Server zbudowana w schemacie galaktyki (opisana w sekcji 4).
4. **Warstwa BI** – raporty i dashboardy w Power BI połączone z hurtownią.

Dane płyną kolejno: źródła → ETL (SSIS) → hurtownia (SQL Server) → Power BI.

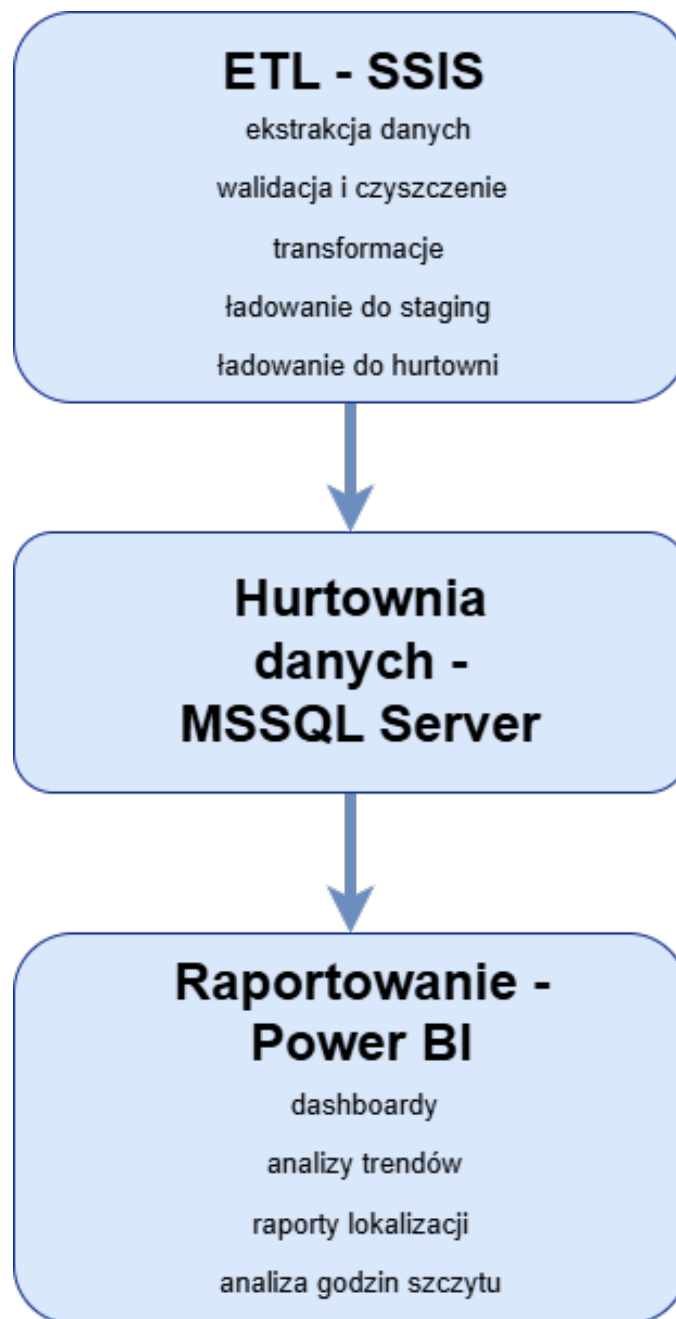


Figure 1: Diagram architektury rozwiązania dla systemu zliczania pieszych.

4 Model hurtowni danych – schemat galaktyki

Hurtownię danych zbudowano w schemacie galaktyki. Model zawiera trzy tabele faktów – **FactPedestrianCount**, **FactWeatherHourly** i **FactDailyTrafficSummary** – które korzystają ze wspólnych wymiarów **DimDate**, **DimTime** i **DimLocation**. Dzięki temu ruch pieszy i warunki pogodowe analizujemy w tym samym kontekście czasu i miejsca. Pogodę zapisano jako tabelę faktów **FactWeatherHourly** z miarami pogodowymi. Wymiary opisują datę, godzinę, lokalizację oraz sensor.

4.3 Typy danych, klucze i wymagalność

Zastosowano typy danych SQL Server: INT, BIGINT, TINYINT, SMALLINT, DATE, DATETIME2, VARCHAR, DECIMAL oraz BIT. Każdy wymiar ma klucz zastępczy (IDENTITY) oraz klucz biznesowy ze źródła. Tabele faktów mają własny klucz zastępczy i klucze obce do wymiarów. Wymiary DimDate i DimTime korzystają z kluczy znaczących. Klucz daty (DateKey) to liczba zapisana jako rok, miesiąc i dzień, np. 15 marca 2024 roku to 20240315. Klucz czasu (TimeKey) to numer godziny doby od 0 do 23, np. godzina 14:00 ma TimeKey równy 14.

4.4 Relacje w modelu

FactPedestrianCount łączy się z DimDate, DimTime, DimSensor i DimLocation. FactWeatherHourly łączy się z DimDate, DimTime i DimLocation. FactDailyTrafficSummary łączy się z DimDate, DimSensor, DimLocation oraz DimTime (godzina szczytu). Wymiary DimDate, DimTime i DimLocation są współdzielone przez tabele faktów. Dodatkowo DimSensor jest powiązany z DimLocation, bo każdy sensor należy do konkretnej lokalizacji.

4.5 Wartości NULL i obsługa braków

W modelu nie dopuszczono wartości NULL – wszystkie kolumny są NOT NULL. Braki obsługujemy wartościami technicznymi: Unknown dla pól tekstowych, 0 lub -1 dla pól liczbowych oraz daty techniczne dla pól dat (1900-01-01 jako brak daty początkowej, 9999-12-31 jako brak daty końcowej). Dla wymiarów SCD 2 dodano rekordy techniczne Unknown z kluczem -1, aby można było podpiąć fakty bez dopasowania klucza biznesowego.

4.6 Obsługa SCD Type 2

Mechanizm SCD Type 2 zastosowano dla DimSensor i DimLocation, bo dane o sensorach i lokalizacjach zmieniają się w czasie. W obu wymiarach dodano kolumny ValidFrom, ValidTo i IsCurrent. Aktualny rekord ma ValidTo = 9999-12-31 oraz IsCurrent = 1. Przy zmianie danych stary rekord jest zamykany (uzupełnienie ValidTo, IsCurrent = 0), a dodawany jest nowy rekord. Dzięki temu zachowujemy historię i analizujemy pomiary zgodnie ze stanem z momentu pomiaru.

4.7 Miary

Miara podstawową jest PedestrianCount – liczba pieszych zarejestrowanych przez sensor w danej godzinie. Sensor zlicza pieszych w dwóch kierunkach ruchu: Direction1Count to liczba pieszych w jednym kierunku, a Direction2Count w przeciwnym; ich suma daje PedestrianCount. Miary dzielimy na bazowe (zapisane wprost w tabelach faktów) oraz wyliczane (liczone w trakcie analizy).

Miary bazowe:

- z FactPedestrianCount: PedestrianCount (liczba pieszych), Direction1Count i Direction2Count (liczba pieszych w każdym z dwóch kierunków ruchu),
- z FactWeatherHourly: TemperatureC (temperatura powietrza), PrecipitationMm (suma opadów w mm), CloudCover (zachmurzenie w procentach), WindSpeed (prędkość wiatru),
- z FactDailyTrafficSummary: DailyTotal (suma pieszych w dniu), AvgPerHour (średnia liczba pieszych na godzinę), PeakHourCount (liczba pieszych w godzinie szczytu).

Miary wyliczane (na podstawie PedestrianCount):

- suma liczby pieszych,

- średnia liczba pieszych,
- maksymalna i minimalna liczba pieszych w godzinie,
- liczba aktywnych sensorów (sensory z `IsCurrent = 1` i statusem aktywnym),
- zmiana liczby pieszych względem poprzedniego miesiąca, wyrażona w procentach,
- zmiana liczby pieszych względem tego samego miesiąca rok wcześniej, wyrażona w procentach.

Miary pogodowe wykorzystujemy w raportach zgodnie z ich charakterem: temperaturę i zachmurzenie uśredniamy w wybranym okresie, a opady sumujemy.

4.8 Atrybuty

Atrybuty to kolumny opisowe w wymiarach, po których filtrujemy i grupujemy dane.

- **DimDate:** FullDate, DayNumber, MonthName, QuarterNumber, YearNumber, DayOfWeekName, WeekOfYear, IsWeekend.
- **DimTime:** HourNumber, PartOfDay.
- **DimLocation:** LocationName, AreaName, Suburb, Street, ZoneType, Latitude, Longitude.
- **DimSensor:** SensorName, SensorStatus, Direction1, Direction2.

4.9 Hierarchie

Hierarchie pozwalają analizować dane na różnych poziomach szczegółowości – od wartości ogólnych po szczegółowe. W modelu zdefiniowano trzy hierarchie:

- **Hierarchia kalendarzowa (DimDate):** Rok → Kwartał → Miesiąc → Dzień.
- **Hierarchia doby (DimTime):** Pora dnia → Godzina.
- **Hierarchia lokalizacji (DimLocation):** Obszar → Dzielnica → Ulica → Lokalizacja.

Wymiar `DimSensor` nie ma naturalnej hierarchii wielopoziomowej – sensory analizuje się poprzez atrybuty (np. status) lub w powiązaniu z lokalizacją.

5 Proces ETL

5.1 Opis procesu ETL i orkestracja

Proces ETL zrealizowano w SQL Server Integration Services (SSIS). Składa się z pakietu głównego `Master.dtsx` oraz pięciu pakietów potomnych, wywoływanych sekwencyjnie z wykorzystaniem komponentu *Execute Package Task* i ograniczeń kolejności (*precedence constraints*). Pakiet główny zawiera mechanizm obsługi błędów (*Event Handler* typu `OnError`), który rejestruje informacje o błędach w tabeli `stg.EtlErrors`:

```
INSERT INTO stg.EtlErrors
(PackageName, TaskName, ErrorMessage, ErrorDate)
VALUES ('Master', 'OnError', 'ETL error occurred', SYSDATETIME());
```

Kolejność wykonania pakietów jest następująca:

1. `01_Extract.dtsx` – ekstrakcja danych z pliku CSV ze zliczeniami pieszych do warstwy staging,
2. `02_Weather.dtsx` – ładowanie danych pogodowych do tabeli faktów `dw.FactWeatherHourly`,
3. `03_Dimensions.dtsx` – ładowanie wymiarów z obsługą SCD Type 2,

4. 04_Facts.dtsx – ładowanie tabel faktów z rozwiązywaniem kluczy zastępczych,
5. 05_QualityControl.dtsx – kontrola jakości danych i rejestracja wyników w etl.LoadAudit.

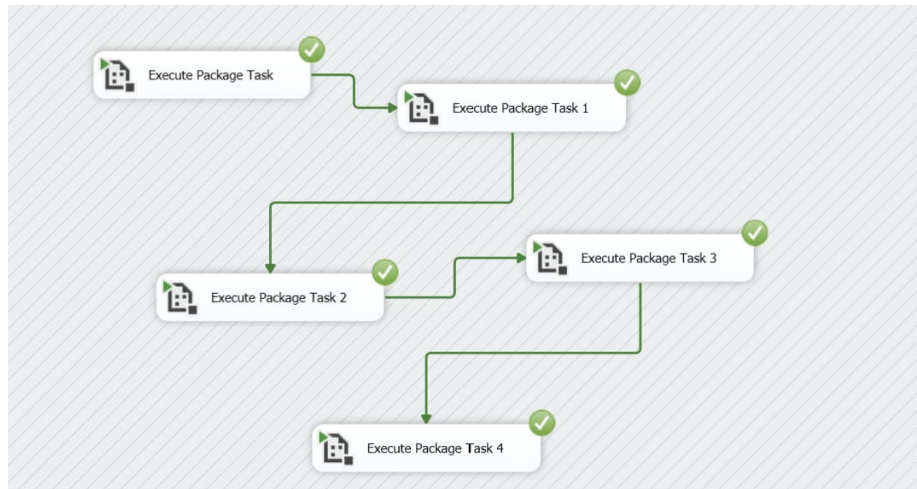


Figure 3: Pakiet główny Master.dtsx z orchestracją procesu ETL.

5.2 Warstwa staging

Przed załadowaniem danych do hurtowni utworzono warstwę staging w schemacie `stg`. Jej zadaniem jest tymczasowe przechowywanie danych źródłowych przed wykonaniem transformacji i walidacji.

Tabela	Opis
<code>stg.PedestrianRaw</code>	Surowe dane o godzinowych zliczeniach pieszych
<code>stg.SensorLocationsRaw</code>	Surowe dane o sensorach i lokalizacjach
<code>stg.WeatherRaw</code>	Dane pogodowe przygotowane do załadowania
<code>stg.EtlErrors</code>	Błędne rekordy i błędy wykryte podczas ETL

5.3 Ekstrakcja danych o zliczeniach pieszych

Pakiet 01_Extract.dtsx ładuje dane z pliku `pedestrian-counting-system-monthly-counts-per-hour.csv` do tabeli `stg.PedestrianRaw` za pomocą przepływu danych z konwersją typów:

Flat File Source → Data Conversion → OLE DB Destination

Komponent *Data Conversion* wykonuje konwersję kolumn: `Sensing_Date` do typu `DATE`, `HourDay`, `Total_of_Directions`, `Direction_1` i `Direction_2` do typu `INT`.

Dane lokalizacji sensorów z pliku `pedestrian-counting-system-sensor-locations.csv` załadowano do tabeli `stg.SensorLocationsRaw` za pomocą drugiego przepływu danych w pakiecie 01_Extract.dtsx:

Flat File Source → Data Conversion → OLE DB Destination

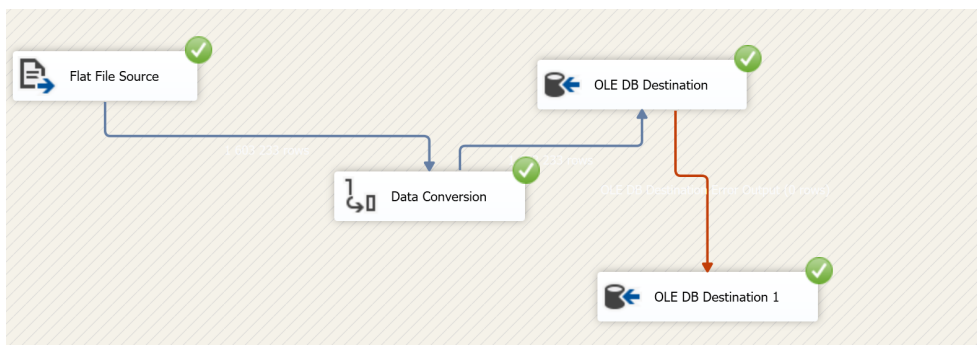


Figure 4: Pakiet 01_Extract.dtsx – ekstrakcja danych.

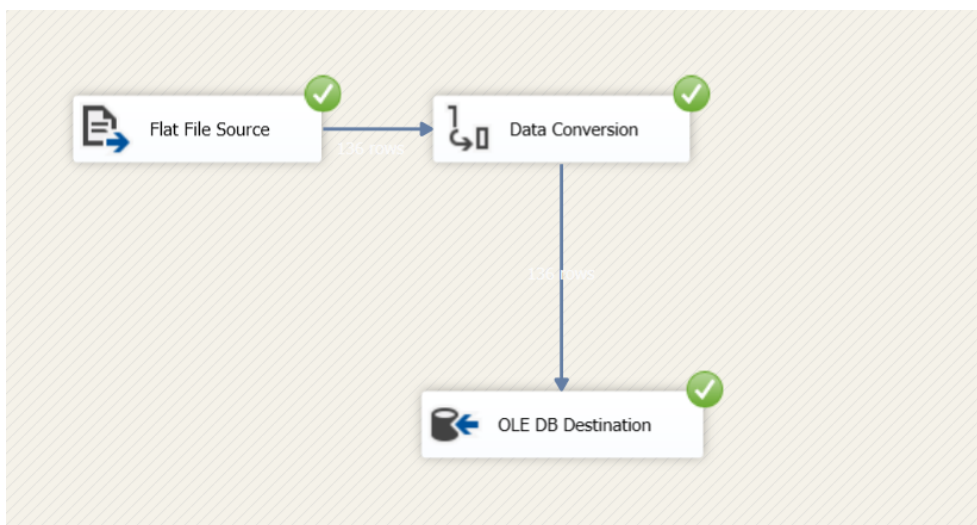


Figure 5: Pakiet 01_Extract.dtsx – ekstrakcja danych.

5.4 Integracja danych pogodowych

Na potrzeby projektu dane pogodowe załadowano do tabeli `stg.WeatherRaw` za pomocą zapytania SQL generującego dane godzinowe dla roku 2024.

Pakiet 02_Weather.dtsx zawiera komponent *Execute SQL Task*, który ładuje dane z `stg.WeatherRaw` do `dw.FactWeatherHourly`:

```

INSERT INTO dw.FactWeatherHourly
(DateKey, TimeKey, LocationKey, TemperatureC,
PrecipitationMm, CloudCover, WindSpeed,
WeatherCode, LoadDate, SourceSystem)
SELECT
CAST(FORMAT(w.Date, 'yyyyMMdd') AS INT),
w.HourOfDay, -1,
w.TemperatureC, w.PrecipitationMm,
w.CloudCover, w.WindSpeed, w.WeatherCode,
SYSDATETIME(), 'Open-Meteo'
FROM stg.WeatherRaw w
WHERE NOT EXISTS (
SELECT 1 FROM dw.FactWeatherHourly f

```

```
WHERE f.DateKey = CAST(FORMAT(w.Date,'yyyyMMdd') AS INT)
AND f.TimeKey = w.HourOfDay);
```

Pierwotnie planowano automatyczne pobieranie danych pogodowych przez komponent *Script Task* w SSIS z wykorzystaniem biblioteki *Newtonsoft.Json*. Implementacja napotkała jednak ograniczenia środowiskowe – SSIS korzysta z kompilatora C# 7.3, który nie obsługuje w pełni zewnętrznych bibliotek NuGet w czasie wykonania. Biblioteka *Newtonsoft.Json.dll* nie mogła zostać załadowana przez silnik SSIS, co skutkowało błędem **Cannot load script for execution**. Z tego powodu zastosowano uproszczone podejście z danymi testowymi.

Docelowo dane pogodowe pobierane będą automatycznie z Open-Meteo Historical Weather API w formacie JSON dla współrzędnych centrum Melbourne (szerokość $-37,8136$, długość $144,9631$) w rozdzielczości godzinowej.

5.5 Ładowanie wymiarów i obsługa SCD Type 2

Pakiet `03_Dimensions.dtsx` ładuje wymiary w kolejności: `DimDate`, `DimTime`, `DimSensor`, `DimLocation`. Każdy wymiar ładowany jest przez osobny komponent *Execute SQL Task*.

DimDate – komponent *Execute SQL Task* pobiera unikalne daty z `stg.PedestrianRaw` i wylicza atrybuty kalendarzowe. **DimTime** – komponent *Execute SQL Task* wstawia 24 rekordy godzinowe z przypisaną porą dnia.

DimSensor z obsługą SCD Type 2 – komponent *Execute SQL Task* wykonuje instrukcję `MERGE`. Dla nowych sensorów wstawiany jest rekord z `IsCurrent = 1`. Przy wykryciu zmiany nazwy sensora stary rekord jest zamykany, a nowy wstawiany.

DimLocation z obsługą SCD Type 2 – analogiczny mechanizm `MERGE` dla wymiaru lokalizacji, z danymi pobieranymi z `stg.SensorLocationsRaw`.

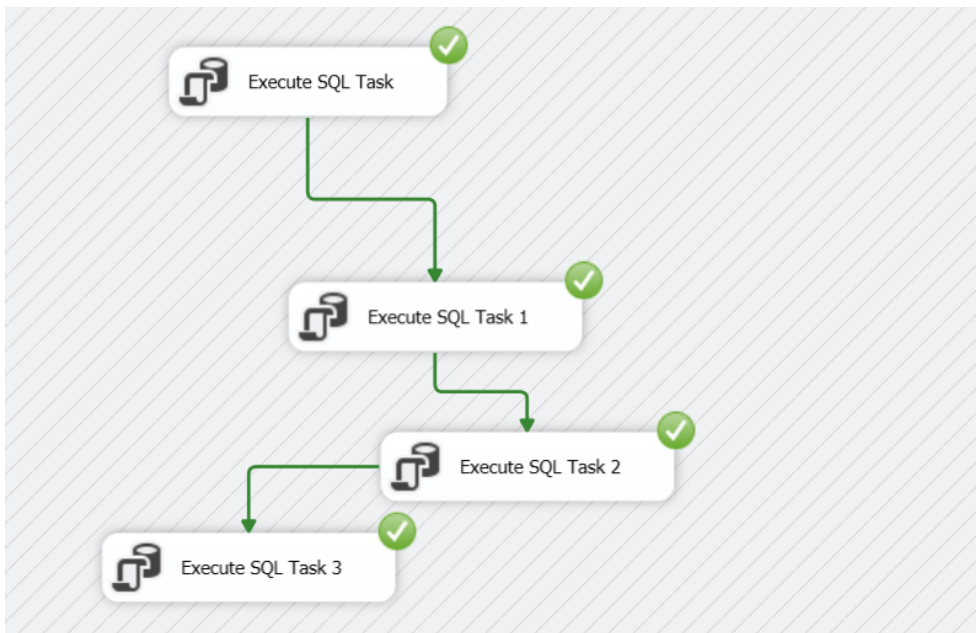


Figure 6: Pakiet `03_Dimensions.dtsx` – ładowanie wymiarów.

5.6 Ładowanie tabel faktów

Pakiet `04_Facts.dtsx` zawiera trzy komponenty *Execute SQL Task* połączone sekwencyjnie.

Execute SQL Task 1 – FactPedestrianCount. Rozwiązywanie kluczy zastępczych przez złączenie z wymiarami po kluczach biznesowych. Ładowanie przyrostowe przez `WHERE NOT EXISTS`.

Execute SQL Task 2 – FactDailyTrafficSummary. Agregacja danych godzinowych do poziomu dziennego.

Execute SQL Task 3 – etl.LoadAudit. Rejestracja wykonania pakietu w tabeli audytu.

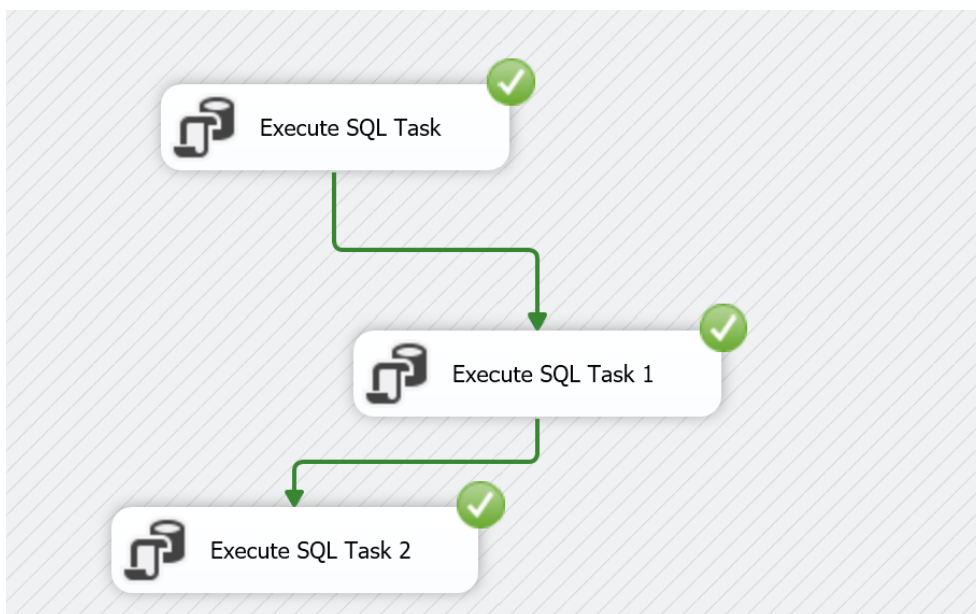


Figure 7: Pakiet 04_Facts.dtsx – ładowanie tabel faktów.

5.7 Ładowanie przyrostowe i audyt

Ładowanie przyrostowe realizowane jest w dwóch scenariuszach:

- **Inicjalizacja** – pierwsze uruchomienie ETL, ładowane są wszystkie rekordy z plików źródłowych,
- **Kolejna iteracja** – kolejne uruchomienie ETL, ładowane są wyłącznie nowe rekordy nieobecne w hurtowni, identyfikowane klauzulą `WHERE NOT EXISTS`.

Tabela `etl.LoadAudit` rejestruje każde uruchomienie procesu ETL. Przechowuje informacje o nazwie pakietu, czasie rozpoczęcia i zakończenia, liczbie przetworzonych i załadowanych wierszy oraz statusie wykonania.

5.8 Obsługa błędów

Błędy wykryte podczas wykonania procesu ETL rejestrowane są w tabeli `stg.EtlErrors`. Mechanizm *Event Handler* typu `OnError` w pakiecie `Master.dtsx` przechwytyje błędy na poziomie całego procesu i zapisuje informacje o nazwie pakietu, zadaniu, treści błędu oraz dacie wystąpienia.

5.9 Kontrola jakości danych

Pakiet `05_QualityControl.dtsx` wykonuje kontrolę jakości danych po załadowaniu hurtowni i rejestruje wyniki w tabeli `etl.LoadAudit`.

5.10 Harmonogram odświeżania

Element procesu	Częstotliwość
Zliczenia pieszych	Miesięcznie
Lokalizacje sensorów	Miesięcznie
Dane pogodowe	Miesięcznie (opcjonalnie częściej)
Wymiary DimDate i DimTime	Po załadowaniu staging
Wymiary DimLocation i DimSensor	Po załadowaniu danych lokalizacji
Tabele faktów	Po załadowaniu wymiarów
Tabela błędów stg.EtlErrors	Przy każdym uruchomieniu ETL

5.11 STTM – Source to Target Mapping

Źródło	Pole źródłowe	Tabela docelowa	Transformacja
ped. CSV	Location_ID	stg.PedestrianRaw.SensorId	Bez transformacji
ped. CSV	Sensing_Date	stg.PedestrianRaw.Date	Konwersja do DATE
ped. CSV	HourDay	stg.PedestrianRaw.HourOfDay	Konwersja do INT
ped. CSV	Total_of_Directions	stg.PedestrianRaw.TotalCount	Konwersja do INT
ped. CSV	Direction_1	stg.PedestrianRaw.Direction1	Konwersja do INT
ped. CSV	Direction_2	stg.PedestrianRaw.Direction2	Konwersja do INT
sensor CSV	Location_ID	dw.DimLocation .LocationBusinessKey	Konwersja do INT
sensor CSV	Sensor_Name	dw.DimSensor.SensorName	Przeniesienie wartości
sensor CSV	Status	dw.DimSensor.SensorStatus	Obsługa SCD 2
sensor CSV	Latitude	dw.DimLocation.Latitude	Konwersja do DECIMAL
sensor CSV	Longitude	dw.DimLocation.Longitude	Konwersja do DECIMAL
SQL (testowe)	dane godzinowe	stg.WeatherRaw.TemperatureC	Losowe wartości DECIMAL
SQL (testowe)	dane godzinowe	stg.WeatherRaw.PrecipitationMm	Losowe wartości DECIMAL
SQL (testowe)	dane godzinowe	stg.WeatherRaw.CloudCover	Losowe wartości SMALLINT
SQL (testowe)	dane godzinowe	stg.WeatherRaw.WindSpeed	Losowe wartości DECIMAL
SQL (testowe)	dane godzinowe	stg.WeatherRaw.WeatherCode	Losowe wartości SMALLINT
ETL tech.	–	dw.DimSensor.ValidFrom	Data uruchomienia ETL
ETL tech.	–	dw.DimSensor.ValidTo	9999-12-31 dla aktualnych
ETL tech.	–	dw.DimSensor.IsCurrent	1 dla aktualnych rekordów

5.12 Analiza jakości danych

Po załadowaniu danych do hurtowni wykonano kontrolę jakości. Sprawdzono braki w kluczowych kolumnach tabeli faktów, wartości ujemne miary głównej, duplikaty na poziomie ziarna oraz spójność liczby wierszy między warstwą staging a hurtownią.

Kontrola	Wynik
Brak DateKey	0 błędów
Brak TimeKey	0 błędów
Brak SensorKey	0 błędów
Brak LocationKey	0 błędów
Ujemny PedestrianCount	0 błędów
Duplikaty ziarna	0 duplikatów
Rekordy w stg.EtlErrors	0 rekordów

Liczba rekordów po wykonaniu procesu ETL:

Tabela	Liczba rekordów
stg.PedestrianRaw	1 603 233
stg.SensorLocationsRaw	136
stg.WeatherRaw	8 760
dw.DimDate	896
dw.DimTime	24
dw.DimSensor	103
dw.DimLocation	103
dw.FactPedestrianCount	1 603 233
dw.FactWeatherHourly	2 048
dw.FactDailyTrafficSummary	70 530

6 Testy funkcjonalne warstwy ETL

Testy funkcjonalne przeprowadzono dla warstwy ETL. Każdy test opisano według schematu: cel, kroki, oczekiwany wynik i potwierdzenie.

Test 1: Spójność liczby wierszy

- **Cel:** Weryfikacja czy liczba wierszy w hurtowni odpowiada liczbie wierszy w pliku źródłowym.
- **Kroki:** Wykonanie zapytania `SELECT COUNT(*)` na tabelach `stg.PedestrianRaw` i `dw.FactPedestrianCount`.
- **Oczekiwany wynik:** Liczby wierszy są równe.
- **Wynik:** Obie tabele zawierają 1 603 233 rekordów. Test zaliczony.

	PedestrianRaw
1	1603233
	FactPedestrianCount
1	1603233

Figure 8: Potwierdzenie spójności liczby wierszy – Test 1.

Test 2: Walidacja wartości miary głównej

- **Cel:** Sprawdzenie czy PedestrianCount nie zawiera wartości ujemnych.
- **Kroki:** `SELECT COUNT(*) FROM dw.FactPedestrianCount WHERE PedestrianCount < 0.`
- **Oczekiwany wynik:** 0 rekordów.
- **Wynik:** 0 rekordów. Test zaliczony.

	UjemnePiesze
1	0

Figure 9: Potwierdzenie braku ujemnych wartości – Test 2.

Test 3: Brak duplikatów ziarna

- **Cel:** Weryfikacja braku duplikatów na poziomie ziarna (DateKey, TimeKey, SensorKey).
- **Kroki:** Grupowanie po kluczach ziarna z warunkiem `HAVING COUNT(*) > 1.`
- **Oczekiwany wynik:** 0 duplikatów.
- **Wynik:** 0 duplikatów. Test zaliczony.

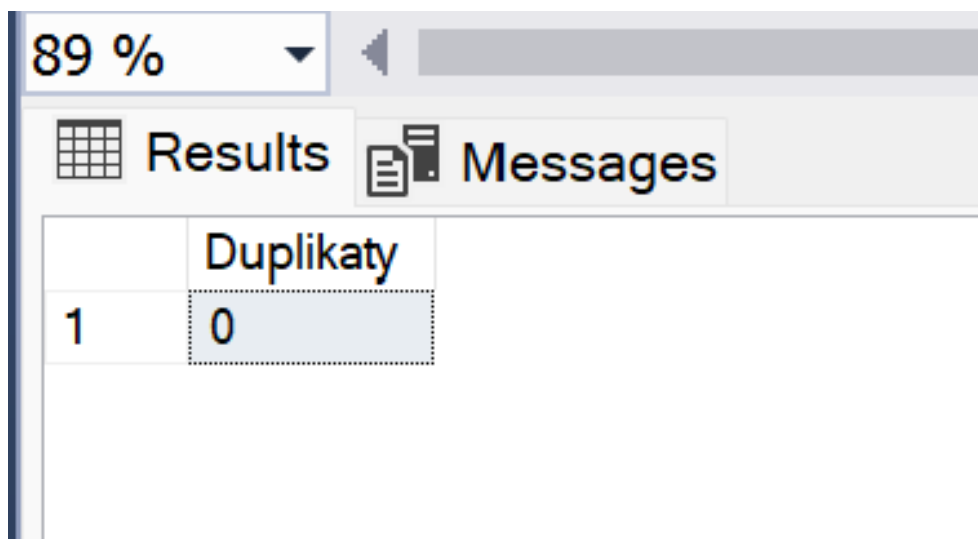


Figure 10: Potwierdzenie braku duplikatów ziarna – Test 3.

Test 4: Poprawność SCD Type 2

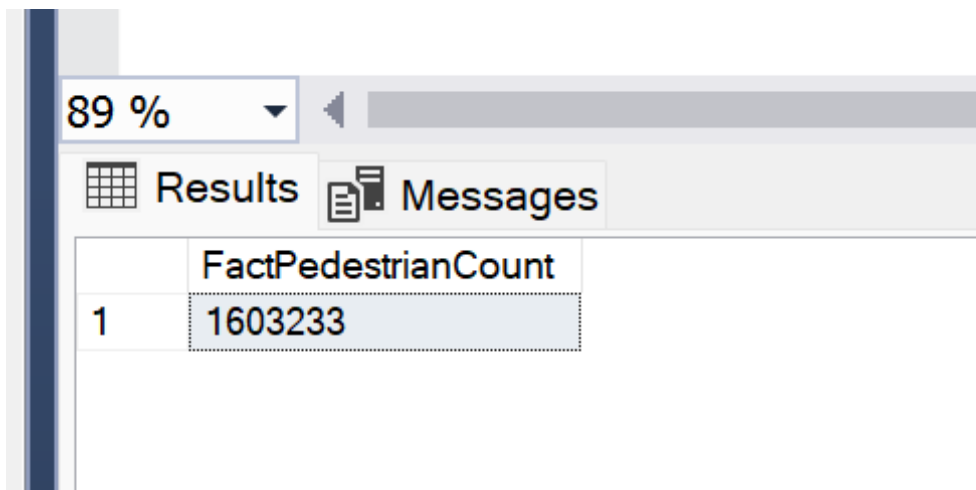
- **Cel:** Weryfikacja czy zmiana atrybutu sensora powoduje zamknięcie starego rekordu i utworzenie nowego.
- **Kroki:** Zmiana statusu sensora o `SensorBusinessKey = 1` z `Active` na `Inactive` i ponowne ładowanie wymiaru.
- **Oczekiwany wynik:** Dwa rekordy: stary z `IsCurrent = 0` i nowy z `IsCurrent = 1`.
- **Wynik:** Rekord historyczny ma `IsCurrent = 0` i zamkniętą datę `ValidTo`. Nowy rekord ma `IsCurrent = 1` i `ValidTo = 9999-12-31`. Test zaliczony.

	SensorBusinessKey	SensorStatus	ValidFrom	ValidTo	IsCurrent
1	1	Active	2026-06-14	2026-06-14	0
2	1	Inactive	2026-06-14	9999-12-31	1

Figure 11: Potwierdzenie poprawności SCD Type 2 – Test 4.

Test 5: Ładowanie przyrostowe

- **Cel:** Weryfikacja czy ponowne uruchomienie ETL nie tworzy duplikatów.
- **Kroki:** Uruchomienie `Master.dtsx` dwukrotnie i porównanie liczby rekordów w `dw.FactPedestrianCount`.
- **Oczekiwany wynik:** Liczba rekordów nie zmienia się przy drugim uruchomieniu.
- **Wynik:** Klauzula `WHERE NOT EXISTS` zapobiega duplikowaniu. Test zaliczony.



	FactPedestrianCount
1	1603233

Figure 12: Potwierdzenie poprawności ładowania przyrostowego – Test 5.

Test 6: Obsługa błędów ETL

- **Cel:** Weryfikacja czy błędy ETL są rejestrowane w `stg.EtlErrors`.
- **Kroki:** Weryfikacja mechanizmu *Event Handler OnError* w `Master.dtsx`.
- **Oczekiwany wynik:** Błędy rejestrowane w `stg.EtlErrors`, ładowanie poprawnych danych nie jest blokowane.
- **Wynik:** Mechanizm *Event Handler* poprawnie rejestruje błędy. Test zaliczony.

7 Warstwa OLAP

Warstwa OLAP to model semantyczny w Power BI zbudowany nad hurtownią danych opisaną w sekcji 4, na bazie tabel faktów i wymiarów ze schematu `dw`.

Relacje. Wszystkie relacje wymiar–fakt są jeden-do-wielu, z filtrowaniem jednokierunkowym (od wymiaru do faktu). `DimDate`, `DimTime` i `DimLocation` są współdzielone przez wszystkie tabele faktów, `DimSensor` – przez `FactPedestrianCount` i `FactDailyTrafficSummary`. Tabele faktów nie są połączone ze sobą bezpośrednio – łączą się wyłącznie przez wspólne wymiary. `FactDailyTrafficSummary` ma dodatkowo relację z `DimTime` po kolumnie `PeakHourKey`, reprezentującą godzinę szczytu w danym dniu.

`DimDate` jest oznaczona jako tabela dat (na podstawie kolumny `FullDate`), co umożliwia obliczenia analizy czasowej (porównania miesiąc do miesiąca i rok do roku). Nazwy miesięcy i dni tygodnia są sortowane kolumnami `MonthNumber` i `DayOfWeekNumber`, dzięki czemu są wyświetlane w porządku kalendarzowym, a nie alfabetycznym.

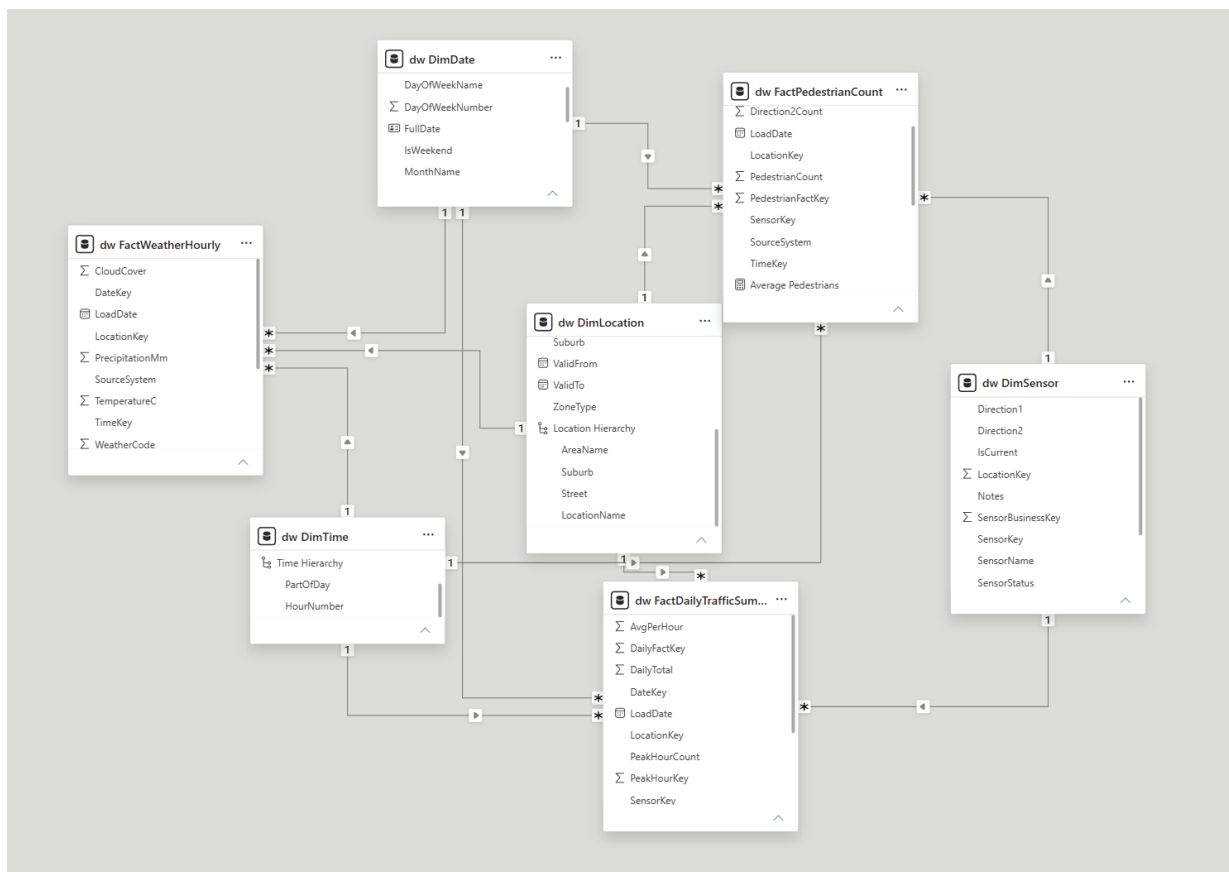


Figure 13: Model semantyczny (warstwa OLAP) z relacjami w Power BI.

Hierarchie. Model wykorzystuje hierarchie zdefiniowane w sekcji 4.9: kalendarzową (Rok → Kwartał → Miesiąc → Dzień), doby (Pora dnia → Godzina) oraz lokalizacji (Obszar → Dzielnica → Ulica → Lokalizacja). Przykład działania hierarchii lokalizacji przedstawiono na rysunku 14.

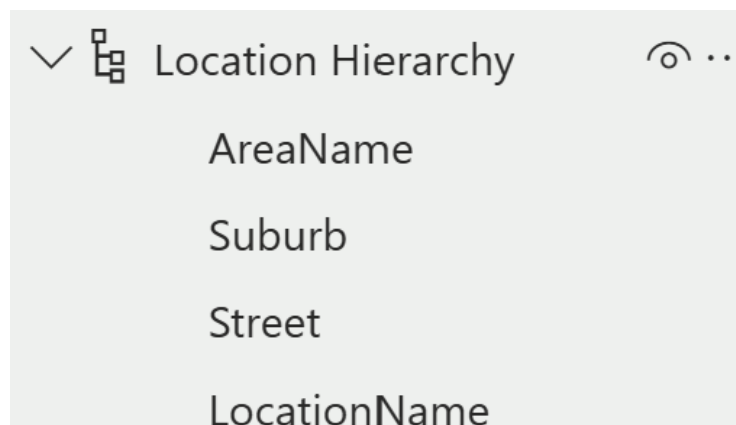


Figure 14: Przykład hierarchii w raporcie Power BI.

7.1 Miary kalkulowane

Na bazie miar opisanych w sekcji 4.7 zdefiniowano w modelu dodatkowe miary kalkulowane w języku DAX:

- ruch pieszy: Total Pedestrians, Average Pedestrians, Maximum/Minimum Hourly Pedestrians, Active Sensors, MoM Change %, YoY Change %;
- pogoda: Average Temperature, Average Cloud Cover, Total Precipitation, Average Wind Speed;
- podsumowania dzienne: Daily Traffic Total, Average Per Hour, Peak Hour Pedestrians, Average Daily Traffic.

8 Realizacja przykładowych raportów

Warstwa raportowa składa się z pięciu stron dashboardu. Przykładowe raporty wyeksportowano do formatu PDF (plik `raporty_BI.pdf` dołączony do dokumentacji).

8.1 KPI i perspektywy

Kluczowe wskaźniki efektywności dostępne w modelu Power BI zdefiniowano jako miary DAX opisane szczegółowo w sekcji ???. Obejmują one: Total Pedestrians, Active Sensors, Avg Daily Traffic, MoM Change% oraz YoY Change%.

W modelu Power BI zastosowano następujące perspektywy analityczne (widoki danych ograniczone do wybranego obszaru tematycznego):

- **Perspektywa ruchu pieszego** – obejmuje FactPedestrianCount, DimDate, DimTime, DimLocation, DimSensor oraz miary: Total Pedestrians, Active Sensors, Avg Daily Traffic, MoM Change%, YoY Change%,
- **Perspektywa pogodowa** – obejmuje FactWeatherHourly, DimDate, DimTime, DimLocation oraz miary: Average Temperature, Total Precipitation, Average Cloud Cover, Average Wind Speed,
- **Perspektywa dzienna** – obejmuje FactDailyTrafficSummary, DimDate, DimLocation, DimSensor oraz miary: Daily Traffic Total, Average Per Hour, Peak Hour Pedestrians.

Perspektywy umożliwiają użytkownikom pracę z uproszczonym widokiem modelu, ograniczonym do tabel i miar istotnych dla danego obszaru analizy.

8.2 Panel główny

Strona pełni rolę punktu wejścia do raportu. Prezentuje pięć wskaźników KPI w formie kart: Total Pedestrians, Active Sensors, Avg Daily Traffic, MoM Change% oraz YoY Change%. Wykres liniowy ilustruje miesięczny trend natężenia ruchu. Slicery umożliwiają filtrowanie po roku i nazwie lokalizacji.

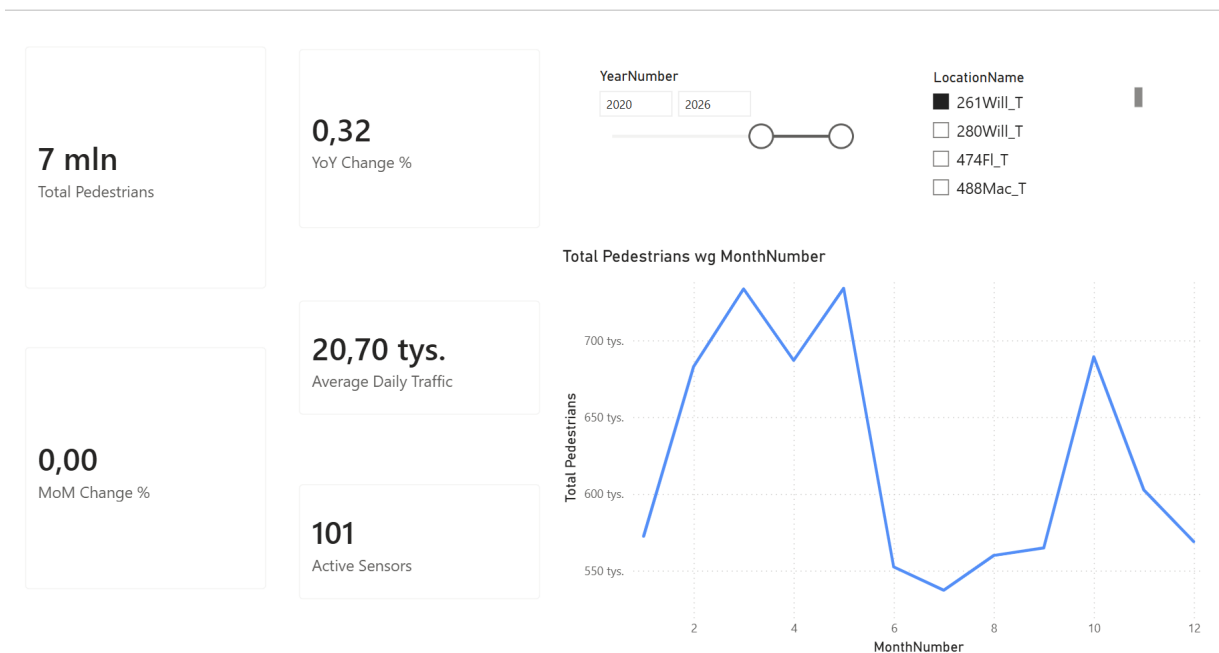


Figure 15: Panel główny – KPI i trend miesięczny.

8.3 Lokalizacja i mapa

Strona prezentuje przestrzenny rozkład natężenia ruchu na mapie bąbelkowej – rozmiar bąbla odpowiada wartości Total Pedestrians dla danej lokalizacji, z wykorzystaniem współrzędnych geograficznych z DimLocation. Tabela poniżej mapy umożliwia porównanie lokalizacji według Total Pedestrians i Active Sensors. Fragmentator miesiąca pozwala analizować zmiany przestrzenne w czasie.

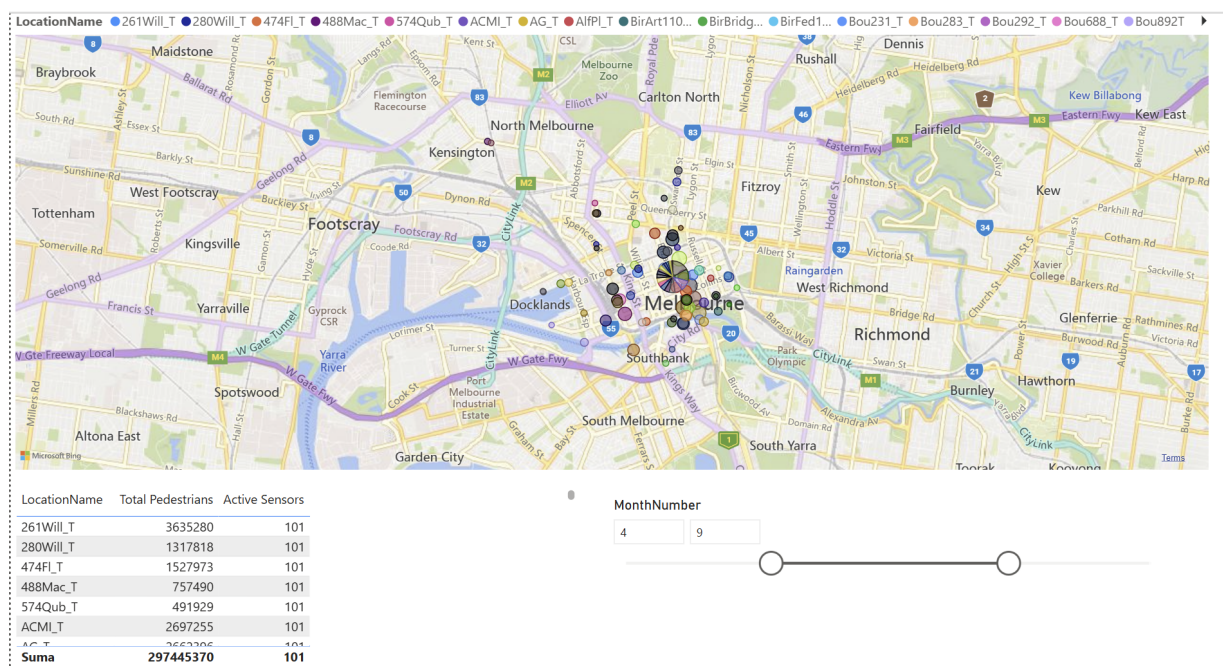


Figure 16: Strona lokalizacji – mapa bąbelkowa i tabela rankingowa.

8.4 Czas i sezonowość

Strona zawiera trzy wizualizacje: wykres słupkowy rozkładu ruchu według dnia tygodnia, wykres liniowy rozkładu godzinowego oraz macierz krzyżową dzień tygodnia × godzina. Strona umożliwia identyfikację godzin szczytu i wzorców sezonowych.

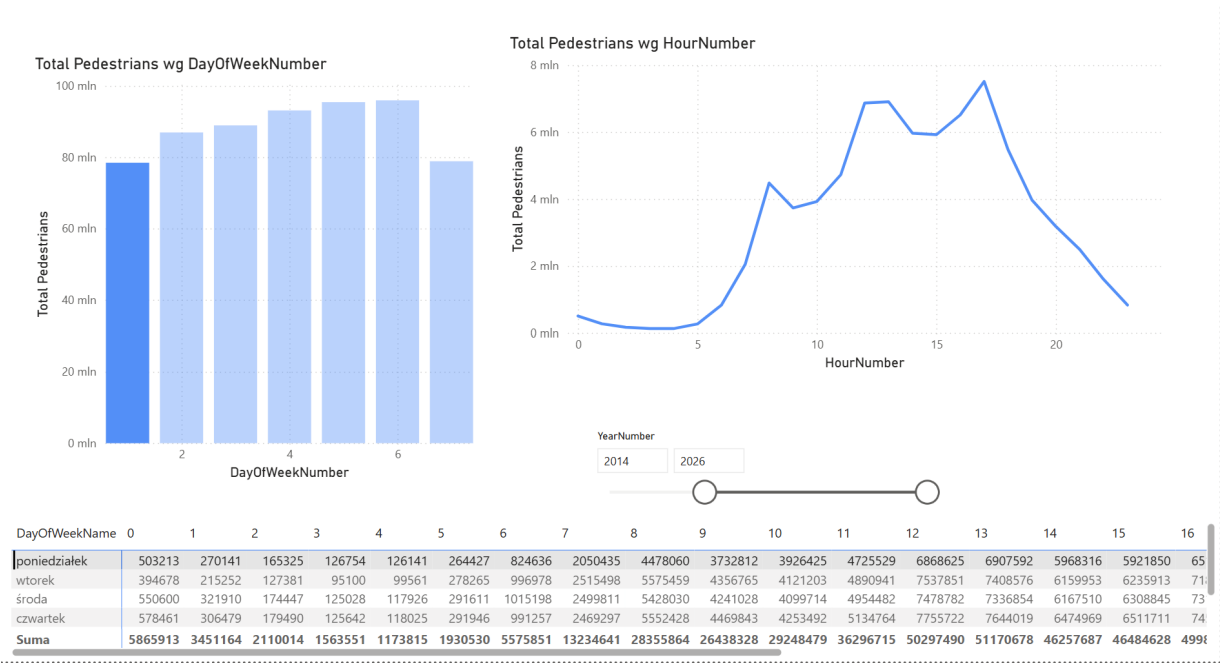


Figure 17: Strona sezonowości – rozkład dzienny, godzinowy i heatmapa.

8.5 Wpływ pogody

Strona analizuje zależność między warunkami pogodowymi a natężeniem ruchu pieszego. Korelacja realizowana jest przez wymiary konformowane DimDate, DimTime i DimLocation, wspólne dla FactPedestrianCount i FactWeatherHourly. Strona zawiera dwa wykresy liniowe na wspólnej osi czasu (MonthNumber): Average Temperature oraz Total Pedestrians, umożliwiające wizualną ocenę korelacji. Wykres słupkowy pokazuje rozkład ruchu według kodu pogody (WeatherCode).

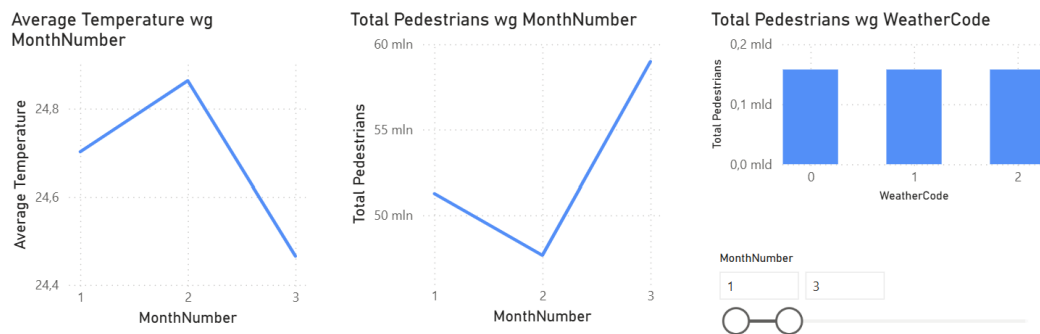


Figure 18: Strona wpływu pogody – temperatura vs ruch pieszzy.

8.6 Porównanie lokalizacji

Strona rankingowa prezentuje Top 10 lokalizacji według Total Pedestrians na wykresie słupkowym poziomym z filtrem Top N. Tabela rankingowa zawiera kolumny: LocationName, Total Pedestrians, Avg Daily Traffic, MoM Change%, YoY Change%. Fragmentator roku umożliwia porównanie rankingów między latami.

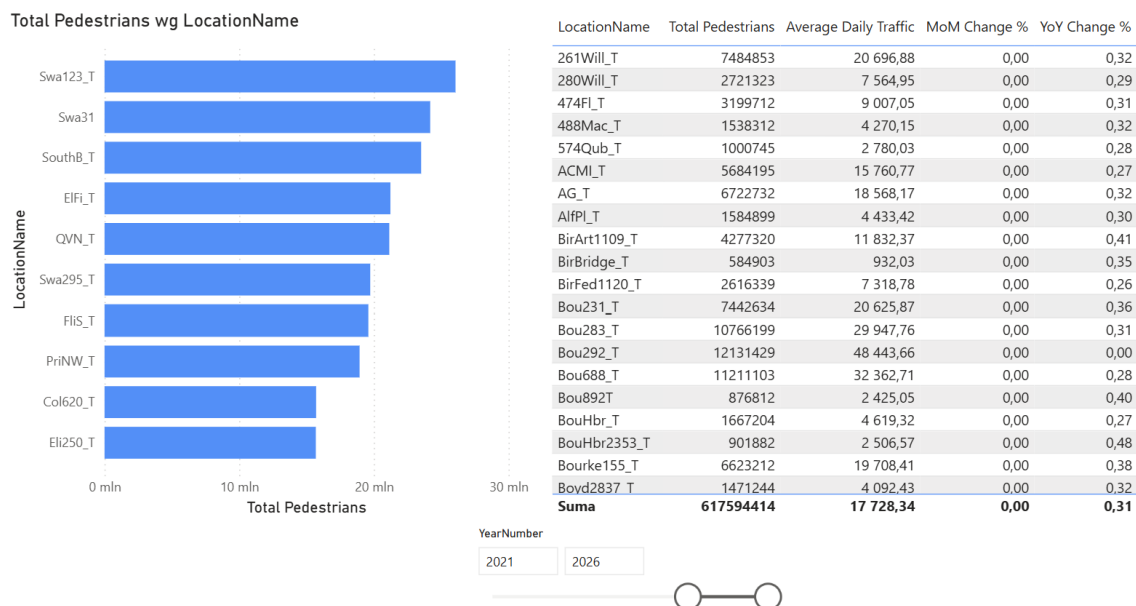


Figure 19: Panel główny – KPI i trend miesięczny.

9 Testy funkcjonalne warstwy BI

Test 1: Poprawność miar KPI

- **Cel:** Weryfikacja zgodności wartości KPI w raporcie z wynikami zapytań SQL na hurtowni.

- **Kroki:** Porównanie wartości Total Pedestrians z raportu z wynikiem zapytania `SELECT SUM(PedestrianCount) FROM dw.FactPedestrianCount`.
- **Oczekiwany wynik:** Wartości identyczne.
- **Wynik:** Wartości zgodne. Test zaliczony.

Test 2: Korelacja pogoda–ruch

- **Cel:** Weryfikacja czy strona pogodowa poprawnie łączy dane z obu tabel faktów przez wymiary konformowane.
- **Kroki:** Wybór miesiąca fragmentatorem – sprawdzenie czy oba wykresy liniowe (temperatura i ruch) filtrują się jednocześnie.
- **Oczekiwany wynik:** Oba wykresy reagują na ten sam filtr miesiąca.
- **Wynik:** Filtry działają przez wspólny wymiar `DimDate`. Test zaliczony.

Test 3: Test end-to-end

- **Cel:** Weryfikacja zgodności wartości raportu z danymi źródłowymi (plik CSV → hurtownia → raport).
- **Kroki:** Wybór konkretnej daty i lokalizacji w raporcie; porównanie Total Pedestrians z wartością w pliku CSV dla tej samej kombinacji.
- **Oczekiwany wynik:** Wartości identyczne.
- **Wynik:** Dane zgodne na wszystkich warstwach. Test zaliczony.

10 Podsumowanie rezultatów projektu

W wyniku realizacji projektu zbudowano kompletny system analityczny obejmujący hurtownię danych w schemacie galaktyki, rozbudowany proces ETL z integracją danych pogodowych oraz warstwę raportową w Power BI.

Z perspektywy biznesowej system dostarcza następujące wartości:

- **Kompletność analityczna** – pięć dashboardów pokrywa kluczowe obszary: trendy globalne, rozkład przestrzenny, sezonowość, wpływ pogody i rankingi lokalizacji,
- **Spójność danych** – wymiary konformowane zapewniają jednoznaczność analiz przekrojowych między ruchem pieszym a danymi pogodowymi,
- **Poprawność KPI** – wartości miar DAX zweryfikowano przez porównanie z wynikami zapytań SQL na hurtowni,
- **Dostarczalność** – raporty wyeksportowano do formatu PDF jako gotowy produkt dla odbiorcy biznesowego.

Analiza danych wykazała, że natężenie ruchu pieszego w Melbourne osiąga szczyty w godzinach 12–17, ze znaczącym spadkiem w nocy. Centrum miasta generuje wielokrotnie wyższy ruch niż lokalizacje peryferyjne. Dane pogodowe obejmują trzy miesiące testowe, co ogranicza wnioski sezonowe – docelowo system powinien operować na pełnym roku kalendarzowym.

Zidentyfikowane ograniczenia obejmują brak odświeżania w czasie rzeczywistym (tryb Import wymaga ręcznego odświeżenia) oraz testowy charakter danych pogodowych z ograniczonym zasięgiem czasowym.

11 Podział pracy w zespole

Projekt realizowany był przez trzyosobowy zespół. Poniżej przedstawiono szczegółowy podział odpowiedzialności.

Oliwia Wójcicka – model hurtowni i warstwa OLAP

- Przeprojektowanie modelu do schematu galaktyki,
- skrypty DDL i wdrożenie w SQL Server Management Studio,
- diagram fizyczny modelu hurtowni,
- definicja miar, atrybutów i hierarchii,
- warstwa OLAP – model semantyczny, perspektywy, hierarchie,
- diagram finalnej architektury rozwiązania.

Daria Bartkowiak – ETL/ELT i analiza jakości danych

- Orchestracja ETL (pakiet master i pakiety potomne),
- integracja Open-Meteo Historical Weather API,
- pełny SCD Type 2 dla DimSensor i DimLocation,
- ładowanie przyrostowe z tabelą `etl.LoadAudit`,
- obsługa błędów i audyt (`stg.EtlErrors`, logowanie),
- analiza jakości danych i aktualizacja tabeli STTM,
- złożenie finalnej paczki kodu.

Karolina Kawulska – warstwa BI, raporty i dokumentacja

- Model biznesowy w Power BI (relacje, hierarchie),
- miary i KPI w języku DAX,
- pięć dashboardów analitycznych,
- eksport raportów do formatu PDF,
- koordynacja montażu i ujednolicenie dokumentacji finalnej,
- przygotowanie prezentacji projektu.

Testy end-to-end oraz przegląd krzyżowy dokumentacji realizowane były wspólnie przez cały zespół.